

Name (Student 1): Devansh Chaturvedi

Roll No: 240340

Name (Student 2): Sthit Pragye_

Roll No: 241045

Course: EE200

Question: 1A

Methodology:

1. The given image is read and converted to 2D NumPy array using Python's cv2 library.
2. Periodic interference, which was introduced in the image to corrupt it, leaves unique fingerprints in the frequency domain. Hence, the image signal needs to be analysed in the frequency domain.
3. The Fourier Transform (F) of the image is calculated w.r.t spatial coordinates x and y using `numpy.fft.fft2()`. This gives us the Fourier transform of the image as a function of u, v. The plot with F(0,0) at the top left corner and F(N-1,M-1) at the bottom right corner where N are the number of points in a single column and M is the number of points in a single row.
4. `numpy.fft.fftshift()` is used to shift the coordinate axes to bring the F(0,0) in the centre of the plot. On going radially outwards from centre, frequencies u and v increase.
5. The frequencies corresponding to noise appear in the form of white patches on the magnitude plot. Their coordinates are identified visually and a small region around these patches are set to 0 to remove the noise.
6. The shifted Fourier plot is reverse shifted to bring back the F(0,0) at the top left corner.
7. Inverse Fourier Transform of this is taken and the real part of the resulting points are considered to form the filtered image.

Answers to Questions asked:

Q. By transforming the image into the frequency domain, can you identify signatures of the unwanted interference?

Ans. Yes, by transforming the image into frequency domain, the unwanted interference appears on the magnitude plot in the form of white patches.

Q. What mathematical equations govern the two-dimensional Discrete Fourier Transform (DFT) and its inverse?

Ans. For an $M \times N$ grayscale image $f(x, y)$, the 2D Discrete Fourier Transform mapping the spatial coordinates (x, y) into the frequency domain (u, v) is given by:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \cdot e^{-i2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right)}$$

The mathematical formula for the 2D Inverse Discrete Fourier Transform used to reconstruct the $M \times N$ image from its frequency domain representation is:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right)}$$

Q. If you plot the magnitude spectrum of the given image in both linear and dB scales, where are the dominant spectral components located?

Ans. The dominant spectral components are located at the centre of image, in rows at 50, 133 and 214 and in the columns numbered 180, 256 and 336.

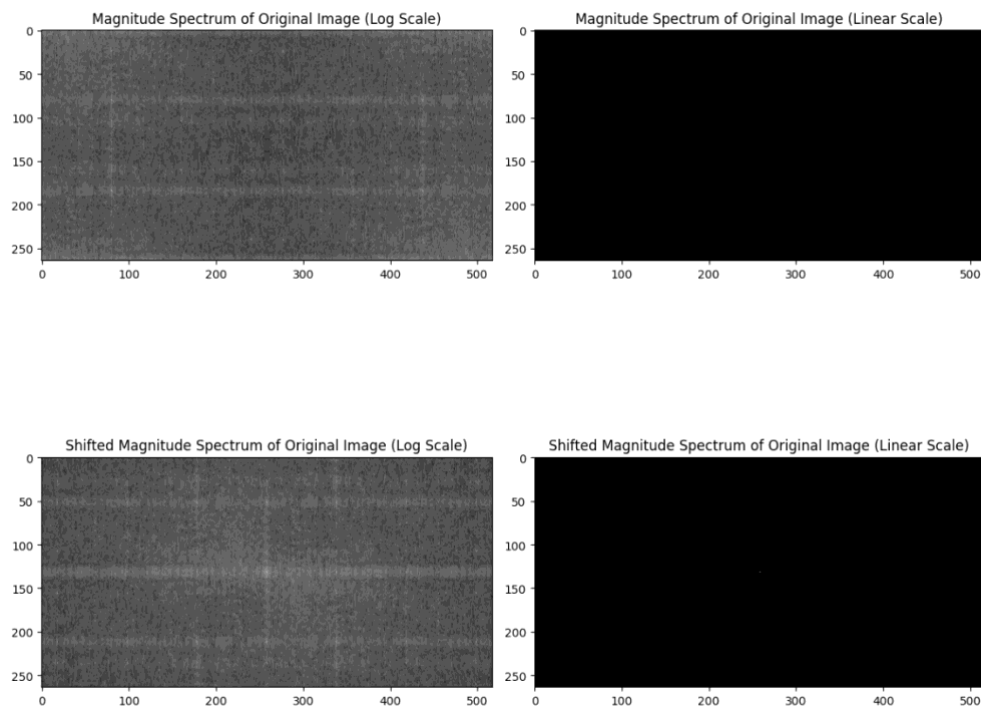
Q. Is the spectrum naturally centred, and if not, how can it be shifted to place the lower frequencies at the centre of the transform?

Ans. No, the spectrum is not naturally centred. To place lower frequencies at the centre of the transform, it is shifted using `numpy.fft.fftshift()`.

Q. The interference has been deliberately introduced in such a way that only a few frequency components are responsible for the corruption. Can you locate these components in the spectrum? How do they differ from the frequencies associated with the actual image content?

Ans. In the frequency spectrum, the noise components appear as small bright patches. The frequencies associated with the actual image content are slightly darker than them. For example, in the shifted magnitude spectrum of the original image (Log scale), the actual image content are the 3 bright horizontal and vertical lines and the noise lies at

their intersection which are unusually brighter (at (133, 180), (133, 256), (133, 336), (50, 180), (50, 256), (50, 336), (214, 180), (214, 256), (214, 336)).



Q. Has the hidden message been successfully recovered?

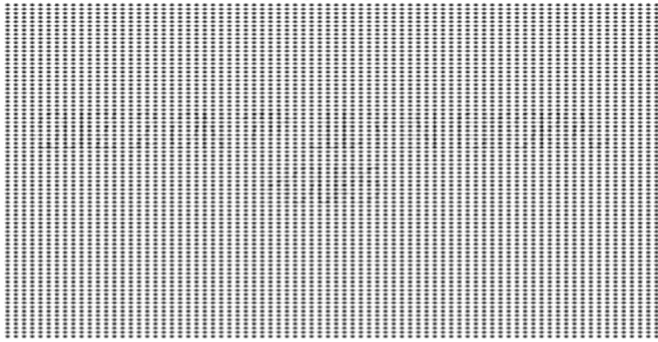
Ans. Yes, the following message has been successfully recovered:

“QUIZ 2 ON 7th JULY IN TUTORIAL HOURS”

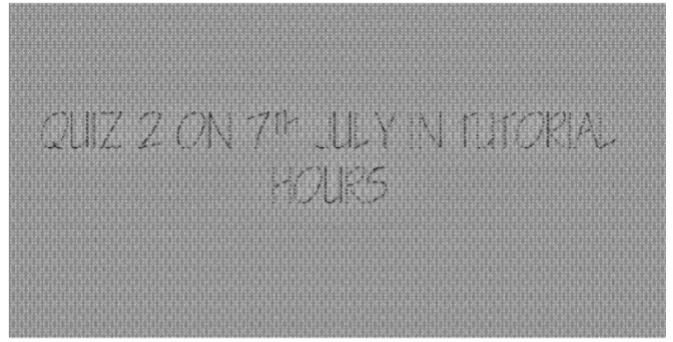
Q. Compare the corrupted image, its frequency spectrum, the filtered spectrum, and the recovered image.

Ans.

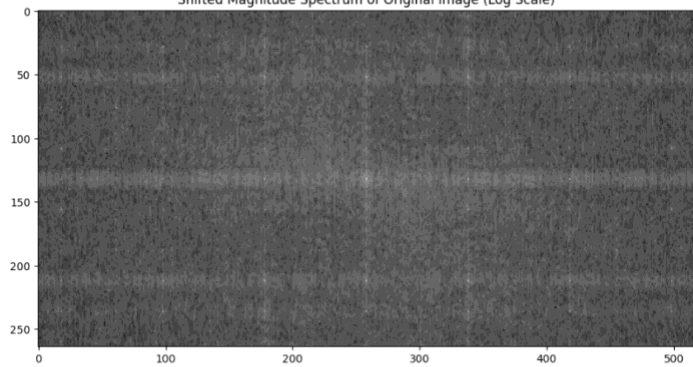
Original Image



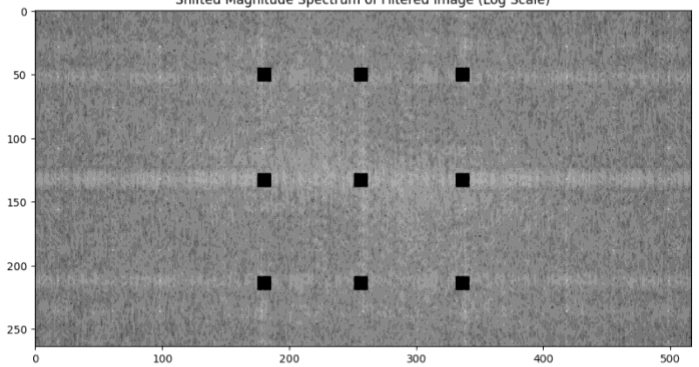
Filtered Image



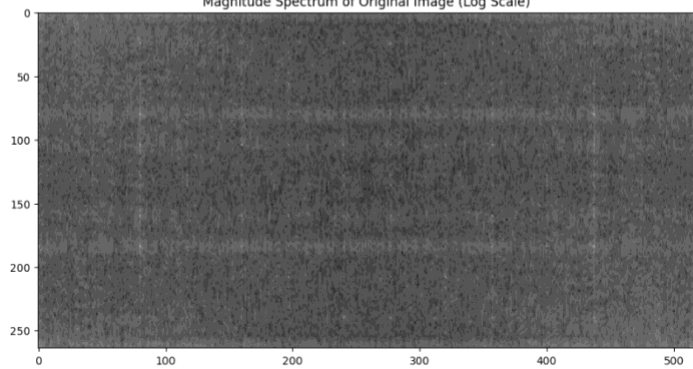
Shifted Magnitude Spectrum of Original Image (Log Scale)



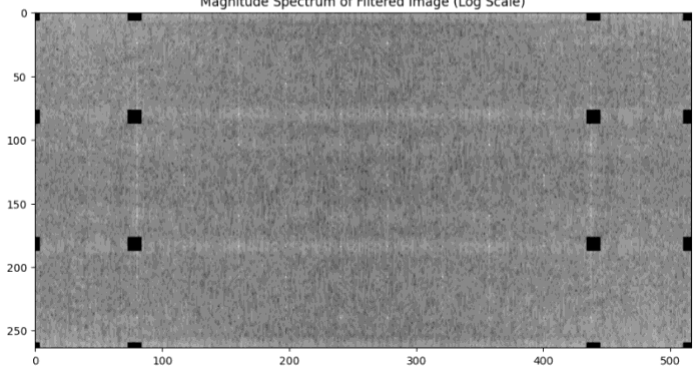
Shifted Magnitude Spectrum of Filtered Image (Log Scale)



Magnitude Spectrum of Original Image (Log Scale)



Magnitude Spectrum of Filtered Image (Log Scale)

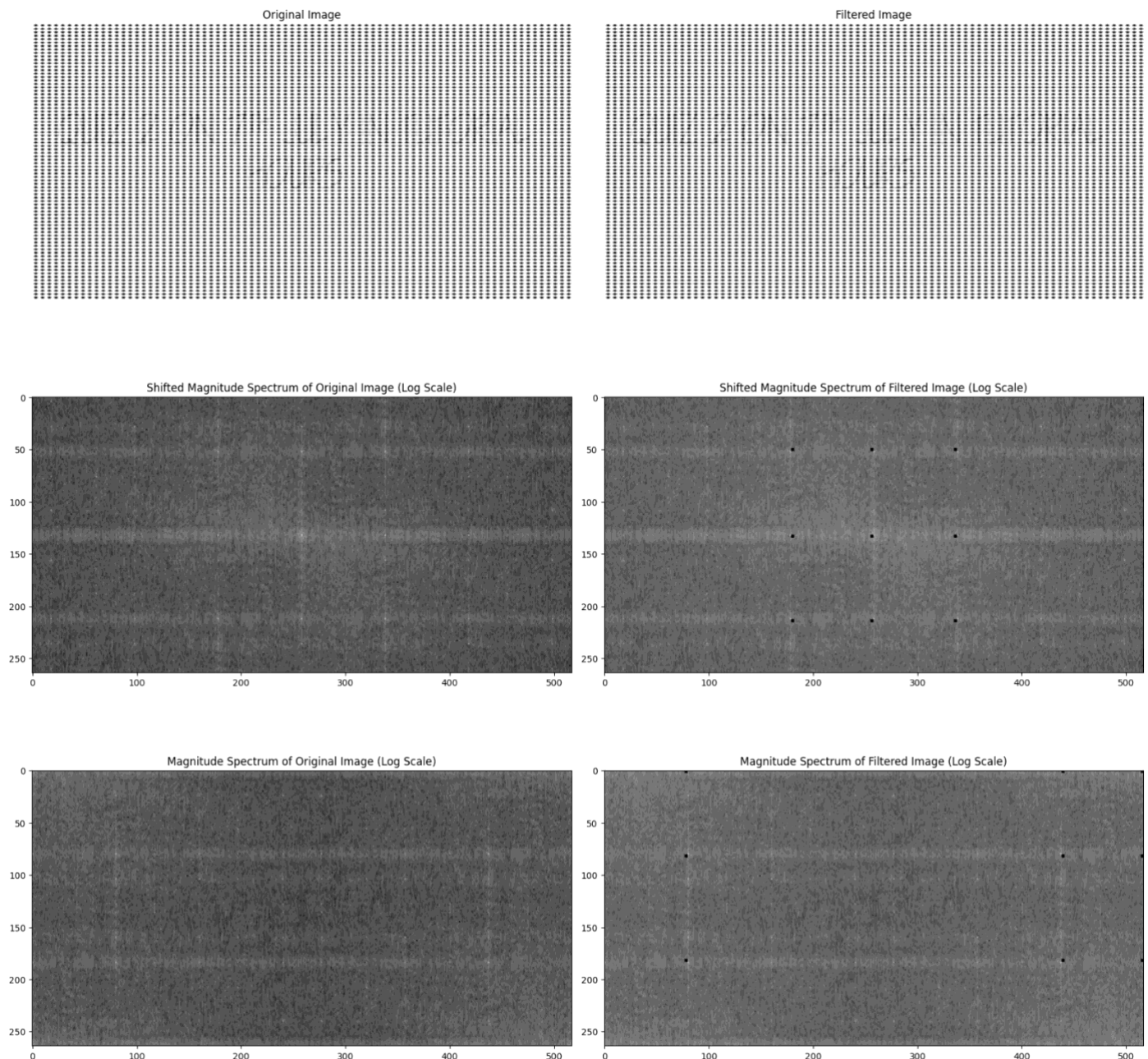


Q. How does the choice of filter parameters affect the quality of re construction? Does removing more frequencies always improve the result, or can it also destroy useful information?

Ans. The number of frequencies removed around the bright patches (determined by r) should neither be too high nor be too low.

- **Case 1:** Too less frequencies are removed ($r < 3$)

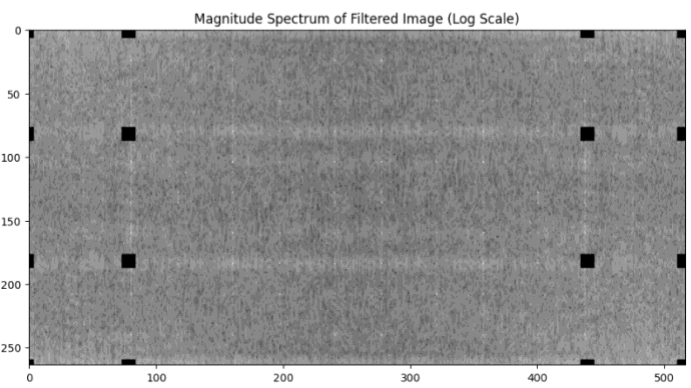
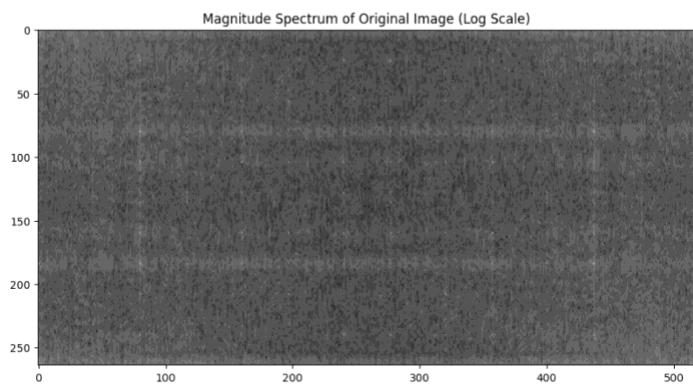
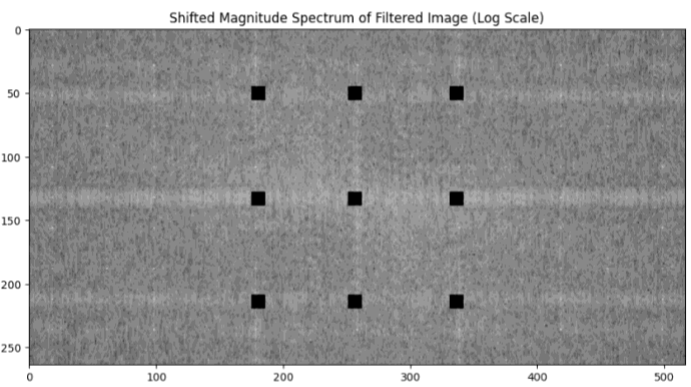
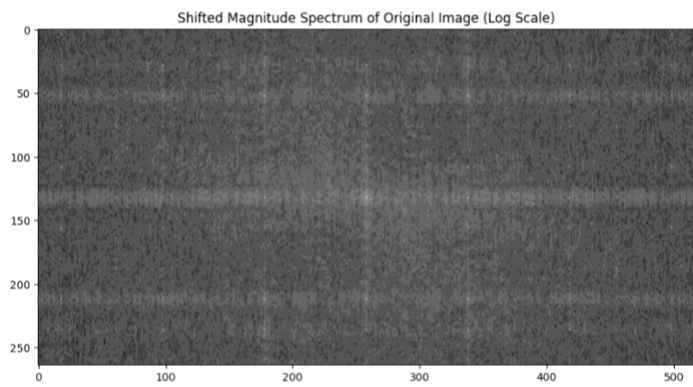
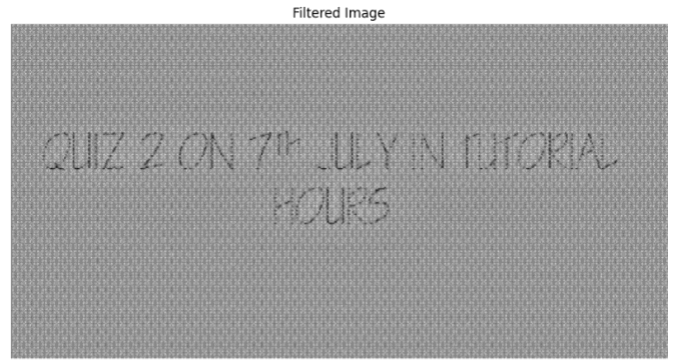
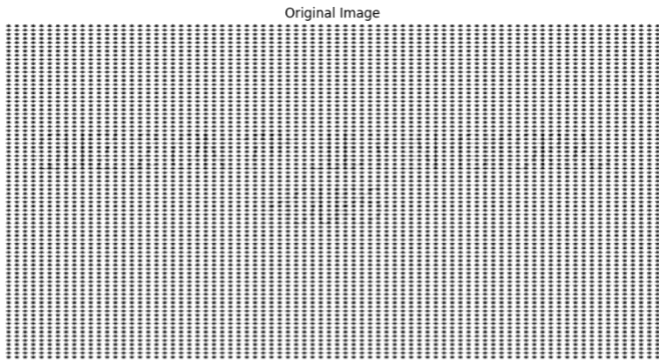
$r=1$



As evident, all the noise frequencies are not completely removed, and the text is still blurred.

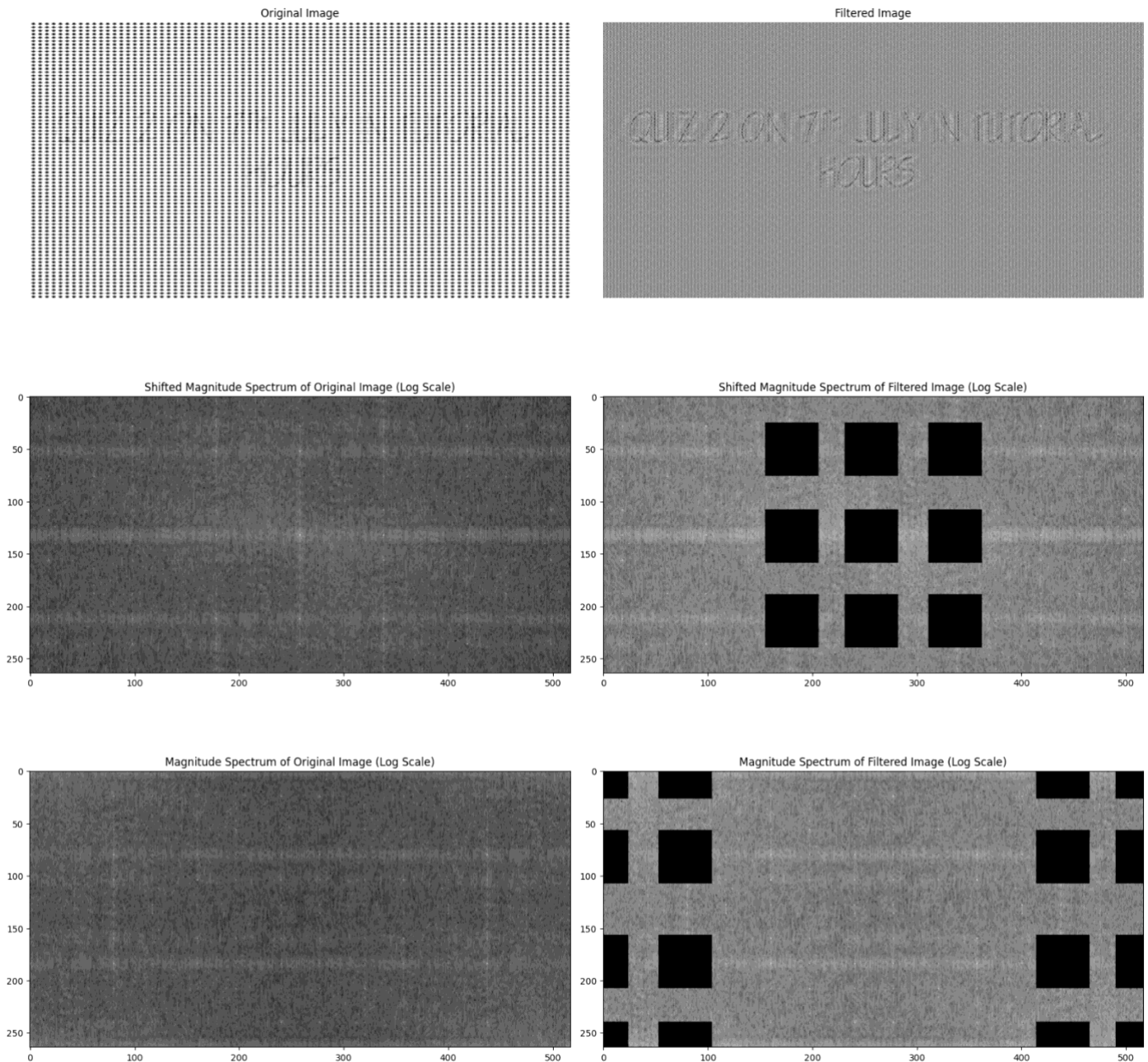
- **Case 2:** Optimal number of frequencies are removed ($6 \Rightarrow r \geq 4$)

$r = 5$



- **Case 3:** Excessive frequencies are removed ($r > 15$)

$$r = 25$$



Huge chunks of the surrounding grey structural bands are swallowed. The frequencies corresponding to the actual image content are also removed in this case. This severely compromises the reconstruction and causes a phenomenon called ringing artifacts (faint, artificial Echo-like waves radiating out from the edges of the text).

Q. Comment on practical applications of frequency-domain image restoration in communication systems, remote sensing, surveillance and medical imaging.

Ans. In communication systems, periodic electronic interference from power lines or circuits creates distracting repeating lines across images. Transforming the image into the frequency domain allows engineers to use sharp notch filters to erase those exact noise frequencies without touching the rest of the picture. Additionally, when wireless signals fade or smudge during transmission, deconvolution filters can invert the channel distortion to restore clean, clear images at the receiver.

In remote sensing and satellite imaging, great distances cause distortions from the environment and movement. Atmospheric turbulence acts like a blur filter that smudges geographic borders, which frequency-domain inverse filters can cleanly sharpen. Since satellites travel at thousands of miles per hour, their movement causes linear motion blur. In the frequency domain, this blur has a highly predictable pattern that processing algorithms can easily reverse to uncover sharp ground details.

In surveillance and security, frequency filtering is used to make blurry faces or license plates readable. Security cameras with bad focus introduce a specific geometric blur that is much easier to un-blur and sharpen in the frequency domain than with standard photo editing tools. For outdoor cameras, homomorphic filtering allows computers to separate low-frequency background fog from the high-frequency details of the actual objects, artificially piercing through thick haze in real time.

In medical imaging, systems like MRI scanners do not actually capture normal spatial pictures. Instead, they directly record raw radio signals from the body inside a frequency grid called k-space, which is the Fourier transform of the patient's anatomy. Doctors use frequency filtering to remove artifacts caused by patient breathing or heartbeats, and to reconstruct the final internal cross-sections before rendering the actual medical scan.

Name (Student 1): Devansh Chaturvedi

Roll No: 240340

Name (Student 2): Sthit Pragye_

Roll No: 241045

Course: EE200

Question: 1B

Methodology:

1. The given image is read using Python's PIL and OpenCV libraries and converted into a NumPy array for further processing.
2. The RGB image is converted into a **grayscale image** using `cv2.cvtColor()`. Since edge detection relies on intensity variations rather than color information, grayscale conversion reduces computational complexity while preserving structural details.
3. To reduce the effect of noise and small intensity fluctuations, a **Gaussian Blur** with a kernel size of **3 × 3** is applied using `cv2.GaussianBlur()`. Smoothing helps suppress unwanted high-frequency noise that may otherwise produce false edges.
4. The smoothed image is processed explicitly using **Sobel operators** to compute the intensity gradients. The first-order spatial derivatives are calculated independently in the horizontal direction (G_x) and vertical direction (G_y) using `cv2.Sobel()` with a 64-bit float depth to accurately capture negative intensity transitions.
5. The absolute values of these directional derivatives are computed using `cv2.convertScaleAbs()` to restore them to standard 8-bit visual ranges.
6. The overall **gradient magnitude** (edge strength) is calculated by blending the absolute horizontal and vertical derivatives equally using `cv2.addWeighted()`.
7. A **binary threshold** of 50 is applied using `cv2.threshold()`. Pixels with a gradient magnitude above 50 are forced to 255 (white edges), and those below 50 are forced to 0 (black background) to filter out weak noise.
8. The final edge map containing the detected boundaries is saved as an output image using `cv2.imwrite()`.

Answers to Questions asked:

Q. What mathematical operation can be used to identify regions where intensity changes rapidly?

Ans. Since an image is a discrete two dimensional signal $I(x,y)$ first order spatial derivatives can be used to locate the points which abrupt changes in

Q. How are derivatives and gradients related to edge detection?

Ans. A derivative mathematically calculates the rate of change. In 2D image processing, the derivative is expressed as a gradient vector, which contains two partial derivatives: $[\partial x/\partial I, \partial y/\partial I]$.

- The gradient magnitude (length of the vector) indicates the sharpness of the edge.
- The gradient direction (angle of the vector) indicates the orientation of the boundary, pointing perpendicular to the edge itself.

Q. Explore the concepts behind the Sobel filter, which is used for 2D convolution operation on images to detect edges.

Ans. The Sobel filter is a discrete differentiation operator. It uses two separate 3×3 matrices (kernels)—one to detect horizontal changes and one for vertical changes. These kernels are convolved (slid across) the image. The unique concept behind the Sobel filter is that it combines a finite difference calculation with a slight perpendicular Gaussian smoothing. This inherent smoothing makes the Sobel operator highly robust to isolated noise pixels compared to a pure mathematical derivative.

Q. How does noise affect the detected boundaries?

Ans. Because edge detection is driven by mathematical derivatives, it is extremely sensitive to noise. High-frequency represents rapid, random changes in pixel intensity. A derivative operator will amplify these random spikes, causing the algorithm to falsely identify them as structural edges. This results in a cluttered image full of non-existing boundaries.

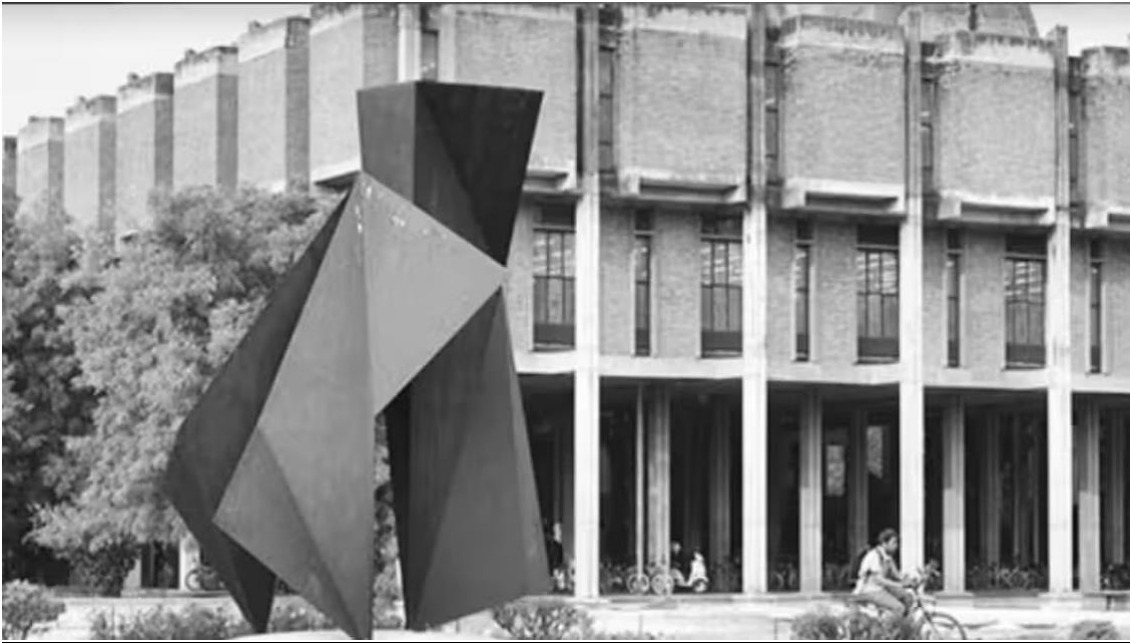
Q. What role does image smoothing play before edge extraction?

Ans. Image smoothing acts as a low-pass filter. By applying a Gaussian blur, small, random high-frequency fluctuations are smoothed out while larger, genuine structural transitions are retained. Smoothing ensures the derivative calculations trigger on true object boundaries rather than artifact noise.

Q. Can weak edges be preserved without introducing excessive noise?

Ans. Using pure binary thresholding (as done in the explicit Sobel code above with a cutoff of 50), preserving weak edges is a strict trade-off: lowering the threshold keeps weak edges but drastically increases noise. However, weak edges can be preserved safely using a technique called **Hysteresis Thresholding** (which is used in the Canny algorithm). Hysteresis uses two thresholds: it keeps all "strong" edges, discards all "noise," and actively preserves "weak" edges only if they are physically connected to a strong structural boundary.

The provided input image:



The processed output:

